

Ag.Cache — Full Folder Details

Shared/Infrastructure/Ag.Cache — the caching library used by EntityManager's demo

Contents

- 1. What this folder is and why it exists
- 2. File-by-file breakdown
- 3. How the files work together — the request flow
- 4. Design decisions and bugs already caught

1. What this folder is, and why it lives here

Ag.Cache is a small, standalone class library — it is the **entire caching feature** for this demo. Nothing in EntityManager.Domain, .Application, or .Infrastructure needed to be rewritten to add caching; those layers were copied from real ag-kit essentially unchanged. Caching was added as a thin layer that plugs into the request pipeline from the outside.

It lives under Shared/Infrastructure (not Shared/BuildingBlocks) because it has a hard dependency on a specific technology — Redis (via the StackExchange.Redis package) — which is exactly what the Infrastructure layer is for in Clean Architecture: concrete technology choices, not business rules.

File	One-line role
Ag.Cache.csproj	Project file — declares the package dependencies (Redis client, Mediator abstractions, DI/config plumbing)
ICacheableQuery.cs	The opt-in interface — a query "raises its hand" to be cached by implementing this
CacheKeyBuilder.cs	Builds the actual cache key string, two ways: manual dictionary, or automatic reflection
CacheKeyIgnoreAttribute.cs	Opt-out marker — excludes a specific property from the automatic (reflection) key-building path
CachingPipelineBehavior.cs	The actual caching logic — intercepts every Mediator request and does the Redis read/write
AgCachingServiceExtensions.cs	One-time startup wiring — connects to Redis and registers it with dependency injection

2. File-by-file breakdown

2.1 — Ag.Cache.csproj

Declares what this project depends on. Nothing in here is demo-specific — every package listed is something real ag-kit already uses elsewhere in the solution (confirmed: `Mediator.Abstractions 3.0.2` matches exactly).

```
<PropertyGroup>
  <TargetFramework>net10.0</TargetFramework>
  <ImplicitUsings>enable</ImplicitUsings>
  <Nullable>enable</Nullable>
</PropertyGroup>

<ItemGroup>
  <PackageReference Include="StackExchange.Redis" Version="2.8.24" />
  <PackageReference Include="Mediator.Abstractions" Version="3.0.2" />
  <PackageReference Include="Microsoft.Extensions.Configuration.Abstractions" Version="10.0.0" />
  <PackageReference Include="Microsoft.Extensions.Configuration.Binder" Version="10.0.0" />
  <PackageReference Include="Microsoft.Extensions.DependencyInjection.Abstractions" Version="10.0.0" />
</ItemGroup>
```

Package	Why it's needed
StackExchange.Redis	The actual Redis client — talks to the Redis server (GET/SET/STREAM commands)
Mediator.Abstractions	Gives access to <code>IPipelineBehavior<,></code> and <code>IRequest<></code> , so this project can plug into Mediator's pipeline without depending on the whole Mediator source generator
Microsoft.Extensions.Configuration.*	Lets <code>AgCachingServiceExtensions.cs</code> read the "Redis" section out of <code>appsettings.json</code>
Microsoft.Extensions.DependencyInjection.Abstractions	Lets <code>AgCachingServiceExtensions.cs</code> register the Redis connection into the DI container

2.2 — ICacheableQuery.cs

This is the **opt-in switch**. A Mediator query that wants caching implements this interface on its record. A query that doesn't implement it is completely invisible to the caching system — nothing about it changes, no performance cost beyond a single type check.

```
public interface ICacheableQuery
{
    string? BuildCacheKey();
}
```

```

    TimeSpan Ttl { get; }
    TimeSpan RefreshWindow { get; }
}

```

Member	Meaning
<code>BuildCacheKey()</code>	Returns the exact Redis key string for this specific call's parameters. Returning <code>null</code> means "don't cache this particular call at all" (see the callout below — this is a deliberate safety feature, not a shortcut).
<code>Ttl</code>	How long a freshly-stored cache entry should live before it expires on its own.
<code>RefreshWindow</code>	How much time-left-on-the-clock triggers a proactive background refresh instead of just serving the (still valid) cached value as-is.

Why `BuildCacheKey()` can return `null` — An earlier version of this demo fell back to a placeholder scope (`ety:default:...`) whenever a required value like `ClientCode` was missing from the request. That was a real bug: nothing watching the database for changes could ever reconstruct that same fake key to invalidate it, so the entry would sit there stale until its TTL happened to expire. Returning `null` instead makes that failure mode structurally impossible — `CachingPipelineBehavior` treats a null key exactly like "this query doesn't implement `ICacheableQuery` at all" and skips Redis completely for that one call.

2.3 — `CacheKeyBuilder.cs`

The single place that decides what a cache key string looks like: `ety:{CLIENTCODE}:{entity}:{operation}:{sorted params}`. Every cacheable query calls into here instead of building its own string by hand, so the format can never drift between query types. It currently offers **two** ways to build the parameter portion of the key:

Build() — the manual, hand-built-dictionary approach

```

public static string Build(
    string clientCode, string entity, string operation,
    IDictionary<string, string?> parameters)
{
    var normalized = parameters
        .Where(p => !string.IsNullOrEmpty(p.Value))
        .OrderBy(p => p.Key, StringComparer.OrdinalIgnoreCase)
        .Select(p => $"{p.Key.ToLowerInvariant()}={p.Value}");

    var paramPart = string.Join("&", normalized);
    return $"ety:{clientCode.ToUpperInvariant()}:{entity}:{operation}:{paramPart}";
}

```

Each caller passes an explicit `Dictionary<string, string?>` naming exactly which of its own properties should scope the cache — this is what most of the 9 cacheable queries in this demo still use today.

- **Null/empty values are dropped** — a request with only `agentKey` set produces a shorter key than one that also has `seoName` set, so two logically different requests never collide.
- **Parameters are sorted by name** — the same logical request always produces the same key, no matter what order its parameters happened to arrive in on the URL.
- **clientId is uppercased** — SQL Server compares it case-insensitively (`'VLA' = 'v1a'`), but a Redis key is just a string with no such rule. Without this, a caller who types `"v1a"` and one who types `"VLA"` would get two totally separate, unrelated cache entries for the same real tenant — and whichever casing the database happens to store is the only one the watcher could ever reconstruct to invalidate. Upper casing here guarantees every caller (and the watcher) always land on the exact same key.

BuildFromObject() — the automatic, reflection-based approach

```
public static string? BuildFromObject(
    string? clientId, string entity, string operation,
    object query, params string[] excludeProperties)
{
    if (string.IsNullOrEmpty(clientId))
        return null;

    var exclude = new HashSet<string>(excludeProperties, StringComparer.OrdinalIgnoreCase);

    var parameters = query.GetType()
        .GetProperties(BindingFlags.Public | BindingFlags.Instance)
        .Where(p => !exclude.Contains(p.Name))
        .Where(p => !InterfaceOwnedProperties.Contains(p.Name))
        .Where(p => p.GetCustomAttribute<CacheKeyIgnoreAttribute>() is null)
        .ToDictionary(p => p.Name, p => p.GetValue(query)?.ToString());

    return Build(clientId, entity, operation, parameters);
}
```

This was added to answer a specific question: *"if a new API parameter gets added later, does the cache key pick it up automatically, or does someone have to remember to add it by hand?"* With `Build()`, the answer is "someone has to remember" — miss that step and two requests that differ only in the new parameter would silently collide onto the same cache key, one of them serving the other's wrong cached result. `BuildFromObject()` reflects over the query record's own public properties instead, so any new property is automatically part of the key with zero changes to that query's `BuildCacheKey()` method.

Because a record's property names get lowercased the same way a hand-written dictionary key would (`AgentKey` → `agentkey`), the two approaches produce **byte-identical keys** for the same query today — this was verified when `GetAgentQuery` was converted, and no Redis flush or watcher change was needed.

InterfaceOwnedProperties — `ICacheableQuery` itself declares two public properties, `Tt1` and `RefreshWindow`, which every cacheable query implements. Reflection has no way to tell those apart from a real filter property like `AgentKey` unless told explicitly — so `CacheKeyBuilder` pre-computes the interface's

own property names once (via reflection over `typeof(ICacheableQuery)` itself) and always excludes them, automatically, for every query that adopts `BuildFromObject()`. See the bug story in section 4 below — this exclusion was added *after* it was caught missing in production testing, not before.

2.4 — `CacheKeyIgnoreAttribute.cs`

```
[AttributeUsage(AttributeTargets.Property)]
public sealed class CacheKeyIgnoreAttribute : Attribute
{
}
```

The general-purpose opt-out valve for the reflection approach. `InterfaceOwnedProperties` handles the one specific, always-true case (every query has `Ttl` / `RefreshWindow`). This attribute handles everything else: if a query record ever grows a property later that isn't meant to scope the cache — e.g. a computed convenience property, an internal correlation ID, anything that isn't really "part of the request's identity" — tagging it `[CacheKeyIgnore]` keeps it out of the key instead of silently fragmenting the cache into more entries than necessary.

2.5 — `CachingPipelineBehavior.cs`

This is where the actual caching decisions happen — a Mediator `IPipelineBehavior<TRequest,TResponse>` that Mediator runs automatically, in front of every request's real handler, no matter which query or command it is.

```
public async ValueTask<TResponse> Handle(
    TRequest request,
    MessageHandlerDelegate<TRequest, TResponse> next,
    CancellationToken cancellationToken)
{
    if (request is not ICacheableQuery cacheable)
        return await next(request, cancellationToken); // not opted in -> pass straight through

    IDatabase db;
    try { db = _redis.GetDatabase(); }
    catch { return await next(request, cancellationToken); } // Redis down -> fail open, don't crash
    the API

    var key = cacheable.BuildCacheKey();
    if (key is null)
        return await next(request, cancellationToken); // missing scoping value -> skip caching
    entirely

    var cached = await db.StringGetWithExpiryAsync(key);

    if (cached.Value.HasValue) // CACHE HIT
    {
        if (cached.Expiry < cacheable.RefreshWindow)
            _ = db.StreamAddAsync("ety:refresh:queue", new[] { new NameValuePair("key", key) });
    }
}
```

```

        return JsonSerializer.Deserialize<TResponse>(cached.Value!, ...!);
    }

    var result = await next(request, cancellationToken);           // CACHE MISS -> run the real handler
    await db.StringSetAsync(key, JsonSerializer.Serialize(result, ...), cacheable.Ttl);
    return result;
}

```

Step by step, every single request that flows through Mediator hits this logic:

1. **Not an `ICacheableQuery`?** Pass through untouched — this is how every write/command in the system stays completely unaffected by caching, with negligible cost (one `is` type check).
2. **Redis unreachable?** "Fail open" — log it and run the real handler as if caching didn't exist, rather than letting a Redis outage take down the whole API.
3. **`BuildCacheKey()` returned null?** Same as step 1 — skip Redis, run the real handler.
4. **Cache HIT?** Deserialize the stored JSON and return immediately — the real handler, repository, and database are never touched for this request.
5. **HIT but close to expiring?** ("close" = less time left than `RefreshWindow`) Still return the still-valid cached value immediately, but also fire-and-forget push the key onto a Redis Stream (`ety:refresh:queue`) so *something else* (a background worker) can refresh it proactively. This is the Stale-While-Revalidate half of the pattern — the current caller never waits on a slow refresh.
6. **Cache MISS?** This is the original, completely unmodified code path — run the real handler exactly as if caching didn't exist, then store its result under the key with the query's own `Ttl` before returning it.

Note about the Mediator pipeline signature — the real `Mediator.Abstractions` library's `IPipelineBehavior<,>.Handle(...)` takes `next` *before* `cancellationToken`. This was confirmed the hard way, by an actual compiler error while building this demo — worth double-checking against whatever Mediator design documentation you're following, since it shows the opposite order.

2.6 — `AgCachingServiceExtensions.cs`

One-time startup wiring, called exactly once from `Program.cs` when the app boots — never during a request.

```

public static IServiceCollection AddAgCaching(this IServiceCollection services, IConfiguration
configuration)
{
    var redisSection = configuration.GetSection("Redis");

    var options = new ConfigurationOptions
    {
        EndPoints = { $"{redisSection["Host"]}:{redisSection["Port"]}" },
        Password = redisSection["Password"],
        Ssl = redisSection.GetValue<bool>("Ssl"),
        AbortOnConnectFail = false
    }
}

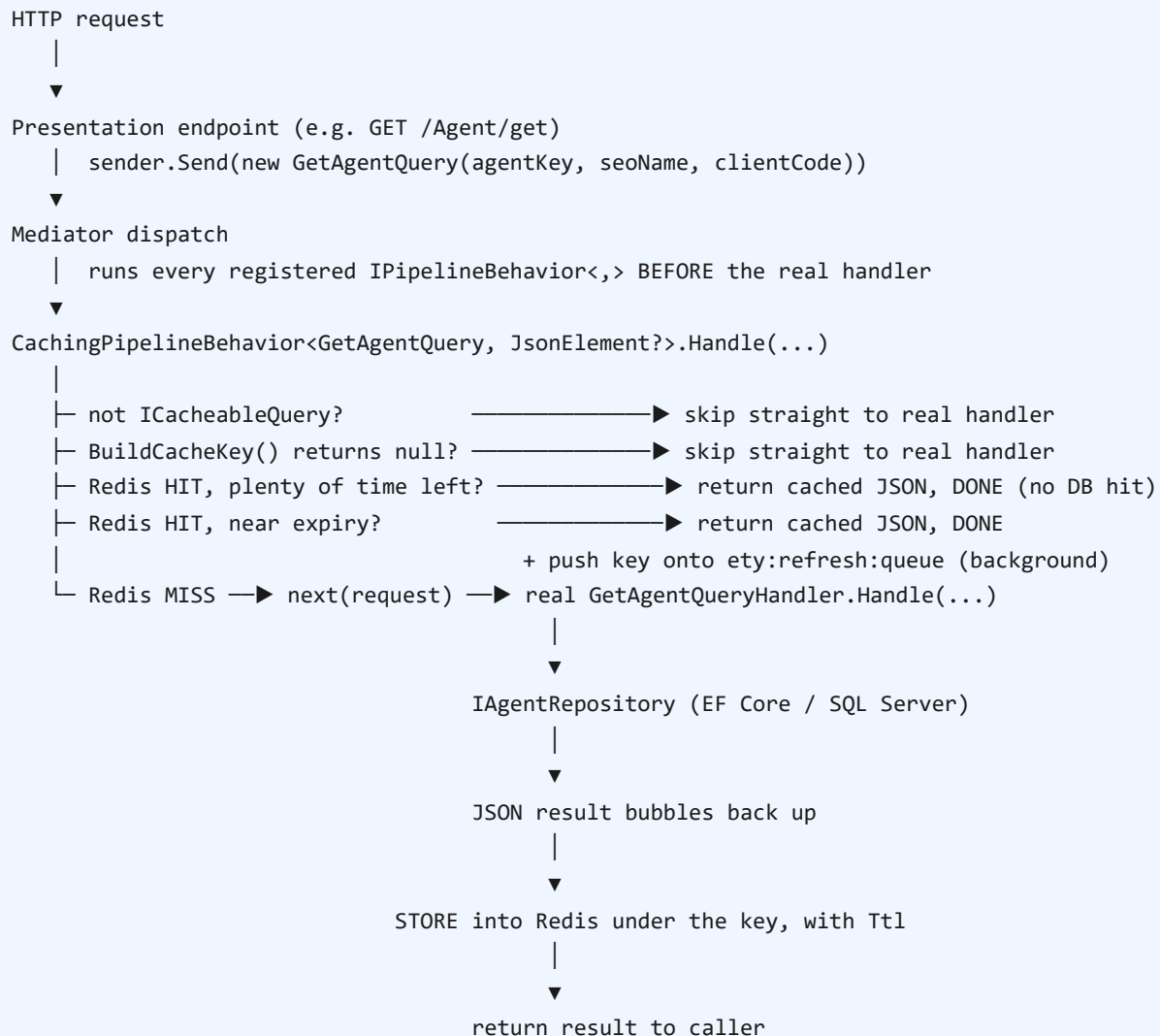
```

```
};

services.AddSingleton<IConnectionMultiplexer>(ConnectionMultiplexer.Connect(options));
return services;
}
```

Reads a "Redis" section from `appsettings.json` (`Host` , `Port` , `Password` , `Ssl`), connects once, and registers that single shared connection as a singleton in DI — `CachingPipelineBehavior` then receives it via constructor injection on every request without reconnecting. `AbortOnConnectFail = false` is deliberate: if Redis happens to be down at the exact moment the API starts up, the API should still start (and simply run uncached / fail-open per request, as described above) rather than crash on boot.

3. How the files work together — the request flow



Two independent processes/services outside the request path also interact with Ag.Cache's keys and streams:

- A **watcher** (in this demo, `Tools/SandboxWatcher`) polls the database directly for changed rows and deletes (invalidates) the matching key using the exact same `CacheKeyBuilder.Build()` logic — it reconstructs the key from the row's own real, canonical `ClientCode` /business key, which only works because `Ag.Cache` never allows a key to be built from anything the watcher couldn't also independently derive from the database row itself.
- A **refresh-queue consumer** drains `ety:refresh:queue` (the stream `CachingPipelineBehavior` pushes onto near expiry) and proactively re-runs the query so the next real caller gets a fast HIT instead of paying for the MISS themselves.

4. Design decisions and a bug already caught here

Real bug caught during testing: when `GetAgentQuery` was first converted to `BuildFromObject()` , `Ttl` and `RefreshWindow` were *not yet* excluded. Because they're public properties on the record (required by `ICacheableQuery`), reflection picked them up exactly like `AgentKey` / `SeoName` , producing keys like: `ety:VLA:agent:get:agentkey=1792&refreshwindow=00:02:00&seoname=susan-harris&ttl=00:10:00` This was caught by inspecting the actual key in RedisInsight, not by any automated test. The fix — `InterfaceOwnedProperties` in `CacheKeyBuilder.cs` — excludes anything declared directly on `ICacheableQuery` automatically, for every current and future query that adopts `BuildFromObject()` , so this class of mistake can't recur per-query.

Trade-off to keep in mind if converting more queries to `BuildFromObject()` : the manual `Build()` approach's risk is *under-inclusion* (forgetting to add a new param — silent stale/wrong cache hits). The reflection approach's risk is the opposite, *over-inclusion* (a future property accidentally becoming part of the key when it shouldn't be — silent cache fragmentation instead of a wrong hit). Neither is strictly safer; `[CacheKeyIgnore]` exists specifically to manage the second risk when it comes up.